

SECUREBANK

Rapport Final — Document des Exigences (Partie 5)

Discussion de la conception sécurisée

Programmation Sécurisée – DevSecOps

DIC2-SSI – École Supérieure Polytechnique de Dakar

Enseignant : Doudou FALL

Équipe de développement

Cheikh Ahmed Tidiane Dieng

Fatou Ndour

Khadidiatou Faye

Mamadou Kone Ndour

Dakar, Juin 2026

1. Introduction

Le présent document constitue le rapport final de la Partie 5 du projet SecureBank. Conformément aux instructions du cours, il s'appuie sur le document d'exigences produit en Partie 2 et y ajoute une discussion détaillée sur la manière dont la conception et l'implémentation répondent aux sept exigences fondamentales de sécurité : autorisation, authentification, audit, confidentialité, intégrité, gestion des secrets et assurance.

SecureBank est une application bancaire simplifiée développée en Java Spring Boot (backend) et React (frontend), déployée localement. L'application couvre les flux principaux d'une banque en ligne : inscription, double authentification, gestion de comptes, virements, journal d'audit, supervision administrative et cycle de vie des comptes. Les deux sprints de développement ont conduit à un système fonctionnellement complet dont la sécurité est assurée par une défense en profondeur cohérente.

2. Autorisation

L'autorisation dans SecureBank est mise en œuvre selon un modèle de contrôle d'accès discrétionnaire (DAC) combiné à un contrôle d'accès basé sur les rôles (RBAC). Le modèle DAC signifie que les ressources sont associées à leurs propriétaires : un client ne peut accéder qu'aux comptes et transactions qui lui appartiennent, quelle que soit la politique globale. Le modèle RBAC, implémenté via Spring Security 6, définit deux rôles distincts : `ROLE_CLIENT` et `ROLE_ADMIN`.

2.1 Politiques DAC et RBAC

Les politiques d'autorisation combinent deux niveaux de contrôle. Au niveau des rôles (RBAC), Spring Security 6 protège les routes par annotation `@PreAuthorize` et par la configuration de `SecurityConfig.java`. Les endpoints `/api/admin/**` sont exclusivement accessibles au rôle `ROLE_ADMIN` ; les endpoints `/api/accounts/**` et `/api/transactions/**` sont accessibles au rôle `ROLE_CLIENT`. Au niveau des données (DAC), `AccountService` et `TransactionService` vérifient systématiquement que l'utilisateur authentifié est bien le propriétaire de la ressource accédée, en comparant l'email extrait du token JWT avec l'email associé au compte en base de données.

2.2 Implémentation

La configuration centrale de Spring Security définit la chaîne de filtres HTTP. Les endpoints publics (inscription, connexion, mot de passe oublié, réinitialisation) sont explicitement déclarés comme accessibles sans authentification dans `SecurityConfig.java`. Tous les autres endpoints requièrent un token JWT valide, vérifié par le filtre `JwtAuthenticationFilter` qui s'exécute avant chaque requête. L'extraction du rôle depuis le token JWT permet à Spring Security d'appliquer les restrictions par rôle sans consulter la base de données à chaque requête.

2.3 Gestion des modifications de politique

Les politiques d'autorisation sont centralisées dans `SecurityConfig.java` pour ce qui concerne les rôles, et dans les couches `Service` pour ce qui concerne la propriété des données. Une modification de politique nécessite donc de modifier uniquement ces deux points d'entrée, sans toucher aux contrôleurs. L'ajout d'un nouveau rôle se ferait en ajoutant une valeur à l'énumération `Role`, en créant les annotations `@PreAuthorize` correspondantes et en configurant Spring Security pour ce rôle. Il n'existe pas actuellement de mécanisme de modification de politique à chaud : tout changement nécessite un redéploiement.

Ressource	Politique applicable	Mécanisme d'application
<code>/api/admin/**</code>	<code>ROLE_ADMIN</code> uniquement	Spring Security — <code>SecurityConfig</code>
<code>/api/accounts/**</code>	<code>ROLE_CLIENT</code> propriétaire	Spring Security + vérif. DAC dans <code>AccountService</code>
<code>/api/transactions/**</code>	<code>ROLE_CLIENT</code> propriétaire	Spring Security + vérif. DAC dans <code>TransactionService</code>
<code>/api/auth/**</code>	Public (sauf change-password)	<code>SecurityConfig</code> — <code>.permitAll()</code>
<code>/api/admin/accounts/deposit</code>	<code>ROLE_ADMIN</code>	Spring Security — <code>@PreAuthorize</code>

3. Authentification

SecureBank implémente une authentification multi-facteurs en trois couches successives. Chaque couche constitue une barrière indépendante : la compromission d'une couche ne suffit pas à obtenir un accès.

3.1 Facteur 1 — Mot de passe haché BCrypt

Lors de l'inscription, le mot de passe de l'utilisateur est haché via `BCryptPasswordEncoder` avec un coût de 12 avant toute persistance en base de données. Ce coût implique $2^{12} = 4\,096$ itérations de la fonction de hachage, ce qui rend les attaques par dictionnaire ou par force brute computationnellement coûteuses. BCrypt intègre un sel aléatoire de 128 bits dans le hash, ce qui garantit que deux mots de passe identiques produisent des hashes différents et neutralise les attaques par tables arc-en-ciel.

La vérification lors de la connexion utilise `passwordEncoder.matches(plainText, storedHash)`, qui recalcule le hash avec le sel extrait du hash stocké et compare les résultats. Le mot de passe en clair n'est jamais persisté ni journalisé.

3.2 Facteur 2 — OTP email (TOTP simplifié)

Après validation réussie du mot de passe, un code OTP à 6 chiffres est généré par `SecureRandom` (générateur cryptographiquement sûr) et envoyé à l'adresse email de l'utilisateur. Ce code est stocké en base de données avec un horodatage d'expiration de 10 minutes. L'accès au compte n'est accordé qu'après soumission du code correct dans ce délai. En cas d'expiration, l'utilisateur doit recommencer le flux de connexion depuis le début. Ce mécanisme protège contre le vol de mot de passe simple et impose que l'attaquant ait simultanément accès au mot de passe et à la boîte email.

3.3 Facteur 3 — Token JWT et mécanismes de session

Après validation de l'OTP, le serveur émet un token JWT signé avec l'algorithme HS256 et la clé secrète chargée depuis la variable d'environnement `{JWT_SECRET}`. Le token contient l'email et le rôle de l'utilisateur dans son payload et expire après 3 600 secondes (1 heure). Côté frontend, le token est stocké dans `localStorage` et transmis dans le header `Authorization : Bearer` à chaque requête. Deux mécanismes complémentaires gèrent la fin de session : la vérification périodique de l'expiration du token toutes les 30 secondes, et la déconnexion automatique après 10 minutes d'inactivité utilisateur détectée par écoute de six événements DOM.

3.4 Mécanismes de protection complémentaires

Le système intègre trois protections supplémentaires contre les attaques sur le flux d'authentification. Le verrouillage après trois tentatives échouées bloque le compte pendant 15 minutes et envoie une alerte email. L'expiration obligatoire du mot de passe après 90 jours (clients) ou 30 jours (administrateurs) force un renouvellement régulier. La détection de connexion depuis une adresse IP non reconnue envoie une alerte email avec l'adresse IP suspecte et invite l'utilisateur à contacter l'administrateur.

Mécanisme	Algorithme / Paramètre	Rôle sécuritaire
Hachage mot de passe	BCrypt, coût 12	Protection contre vol de BDD + force brute
OTP connexion	SecureRandom 6 chiffres, exp. 10 min	2ème facteur — possession email

Token de session	JWT HS256, exp. 1h	Autorisation sans état côté serveur
Inactivité	Timer 600 000 ms, reset sur 6 events DOM	Fermeture session non surveillée
Verrouillage	3 tentatives, 15 min	Protection contre force brute
Expiration MDP	90j client / 30j admin	Rotation obligatoire des secrets

4. Audit

La responsabilité des actions utilisateurs est assurée par un journal d'audit persisté en base de données avec chaînage cryptographique HMAC-SHA256. Ce mécanisme garantit non seulement la traçabilité mais aussi l'intégrité du journal : toute modification ou suppression d'une entrée rompt la chaîne et est détectable.

4.1 Fonctionnement du chaînage HMAC

Chaque entrée du journal contient six champs : l'identifiant de l'action, l'email de l'utilisateur, le type d'action, un horodatage, les détails de l'action, et une valeur HMAC. Le HMAC de chaque entrée est calculé sur la concaténation de son contenu propre et du HMAC de l'entrée précédente, formant une chaîne : $\text{HMAC}_n = \text{HMAC-SHA256}(\text{clé}, \text{action}_n || \text{HMAC}_{\{n-1\}})$. Ce mécanisme est similaire en principe à la structure d'une blockchain simplifiée.

La clé HMAC est distincte de la clé JWT et de la clé AES, chargée depuis la variable d'environnement `#{HMAC_SECRET}`. Cette séparation des clés par usage respecte le principe de moindre privilège cryptographique.

4.2 Actions journalisées

Le journal enregistre l'ensemble des actions à impact sécuritaire ou financier : inscription, connexion réussie et échouée, verrouillage de compte, envoi d'OTP, changement de mot de passe, réinitialisation de mot de passe, virement, dépôt, demande et traitement de suppression de compte, connexion depuis une IP suspecte, et expiration de mot de passe. Chaque entrée identifie l'acteur (email utilisateur ou identifiant admin), l'action effectuée, l'horodatage UTC et les paramètres pertinents.

4.3 Accès au journal

L'administrateur peut consulter le journal d'audit depuis le tableau de bord administrateur. L'accès est restreint au rôle `ROLE_ADMIN` par Spring Security. Les entrées sont présentées en ordre chronologique inverse. La vérification de l'intégrité de la chaîne peut être effectuée par `AuditService.verifyChain()` qui recalcule chaque HMAC et le compare à la valeur stockée.

5. Confidentialité

La confidentialité des données est assurée à deux niveaux : les données en transit sont protégées par TLS, et les données sensibles au repos sont chiffrées par AES-256-GCM.

5.1 Chiffrement en transit — TLS

Toutes les communications entre le navigateur et le serveur backend sont acheminées via HTTPS (TLS 1.2 minimum). Dans le contexte de déploiement local du projet, le certificat TLS est auto-signé. En production, un certificat émis par une autorité de certification reconnue (Let's Encrypt par exemple) serait requis. TLS garantit la confidentialité des données en transit, l'authenticité du serveur et l'intégrité des échanges contre les attaques de type man-in-the-middle.

5.2 Chiffrement au repos — AES-256-GCM

Les données sensibles persistées en base de données sont chiffrées par le composant `AesEncryptor.java`. L'algorithme retenu est AES en mode GCM (Galois/Counter Mode) avec une clé de 256 bits. Ce mode offre simultanément la confidentialité (chiffrement par flux de compteur) et l'authenticité (tag d'authentification de 128 bits), ce qui le distingue des modes comme CBC qui n'offrent que la confidentialité. Les champs chiffrés incluent les numéros de compte et les informations sensibles des transactions.

Chaque chiffrement génère un vecteur d'initialisation (IV) aléatoire de 96 bits via `SecureRandom`, concaténé au ciphertext avant stockage. L'utilisation d'un IV unique par opération est essentielle pour la sécurité du mode GCM : réutiliser le même IV avec la même clé permettrait à un attaquant de récupérer le masque de chiffrement. La clé AES de 256 bits est chargée depuis la variable d'environnement `${AES_KEY}` et n'est jamais persistée dans le code source.

5.3 Minimisation des données dans le module admin

Les statistiques exposées à l'administrateur dans le tableau de bord sont délibérément agrégées : total des montants virés, nombre total de transactions, nombre de clients actifs. Ces indicateurs ne révèlent aucune information sur l'identité des parties ou le détail des opérations individuelles. Cette approche applique le principe de minimisation des données au niveau de l'interface administrateur.

6. Intégrité

L'intégrité des données est maintenue par trois mécanismes complémentaires opérant à différents niveaux de la pile applicative.

6.1 Transactions ACID

Toutes les opérations financières (virements, dépôts, suppressions de compte) sont exécutées dans des transactions de base de données annotées `@Transactional`. Les propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité) garantissent qu'une opération financière ne peut pas se terminer dans un état partiel : soit le débit et le crédit sont tous deux effectués, soit aucun des deux ne l'est. En cas d'exception à tout moment de l'opération, la transaction est rollbackée et la base de données revient à son état précédent.

6.2 Chaînage HMAC du journal d'audit

Comme décrit dans la section Audit, le journal est structuré en chaîne HMAC-SHA256. Cette structure garantit non seulement la traçabilité mais aussi l'intégrité historique : il est impossible de modifier, supprimer ou insérer rétroactivement une entrée sans que la vérification de la chaîne ne le détecte. Ce mécanisme transforme le journal d'audit en registre tamper-evident.

6.3 Validation des entrées

Toutes les entrées utilisateur sont validées côté backend avant traitement. Les montants financiers sont vérifiés comme strictement positifs. Les numéros de compte sont vérifiés comme appartenant à l'utilisateur authentifié. Les adresses email sont validées par expression régulière. Les mots de passe sont soumis aux règles de complexité. Cette validation systématique protège contre les injections et les états incohérents en base de données.

7. Gestion des secrets

Cette section décrit la génération, le stockage, la mise à jour et la suppression de chaque catégorie de secret utilisé dans SecureBank, ainsi que les hypothèses sur les fonctionnalités du système d'exploitation.

7.1 Mots de passe utilisateurs

Les mots de passe sont générés par les utilisateurs eux-mêmes, sous contrainte de complexité : minimum 8 caractères, au moins un chiffre, au moins un caractère spécial. Ils ne sont jamais stockés en clair : seul le hash BCrypt (coût 12) est persisté en base de données. La mise à jour d'un mot de passe exige la saisie de l'ancien mot de passe et la conformité du nouveau aux règles de complexité. Le champ `passwordChangedAt` est mis à jour à chaque changement. La suppression est implicite à la suppression du compte utilisateur. Aucun mot de passe en clair ne transite par les logs.

7.2 Clés cryptographiques et secrets applicatifs

Quatre secrets applicatifs sont utilisés : la clé secrète JWT (HS256, recommandation : 256 bits minimum, encodée en Base64), la clé AES (256 bits, encodée en Base64), la clé HMAC-SHA256 du journal d'audit (256 bits minimum), et le mot de passe de la boîte email d'envoi. Ces quatre secrets sont chargés exclusivement depuis des variables d'environnement via la syntaxe `${VAR}` de Spring Boot, documentées dans le fichier `.env.example`.

- Génération : les clés sont générées avant le premier déploiement avec un générateur cryptographiquement sûr. Pour AES-256, la commande `openssl rand -base64 32` produit une clé de 256 bits encodée en Base64. Pour JWT et HMAC, une longueur identique est recommandée.
- Stockage : les clés sont stockées dans un fichier `.env` local, ajouté au `.gitignore` et jamais commité. En environnement de production, un gestionnaire de secrets (HashiCorp Vault, AWS Secrets Manager) serait préférable.
- Mise à jour : une rotation de clé nécessite d'arrêter l'application, de mettre à jour la variable d'environnement et de redémarrer. Pour la clé JWT, tous les tokens actifs sont immédiatement invalidés. Pour la clé AES, les données chiffrées avec l'ancienne clé doivent être rechiffrées.
- Suppression : à la fin du projet, les fichiers `.env` sont supprimés manuellement. Les variables d'environnement ne persistent pas au-delà de la session de déploiement.

7.3 Tokens temporaires

Deux catégories de tokens temporaires sont utilisés. Les OTP de connexion (6 chiffres, SecureRandom, expiration 10 minutes) sont stockés en base de données sous forme hachée et invalidés après usage unique. Les tokens de réinitialisation de mot de passe (8 caractères alphanumériques majuscules, `UUID.randomUUID()`, expiration 30 minutes) sont stockés hachés dans les champs `resetPasswordToken` et `resetPasswordTokenExpiry` et invalidés après usage. Dans les deux cas, la réponse serveur ne révèle pas l'existence ou non de l'email en base de données, prévenant ainsi l'énumération des comptes.

7.4 Hypothèses sur le système d'exploitation

Le projet suppose que le système d'exploitation hôte assure l'isolation des processus et la protection de la mémoire (pas d'accès direct à la mémoire d'un processus par un autre). Les variables d'environnement sont supposées lisibles uniquement par le processus qui les a reçues et ses descendants. Le système de fichiers est supposé configurable pour restreindre l'accès au fichier `.env` au seul utilisateur propriétaire du

processus (chmod 600 sous Linux/macOS). Ces hypothèses sont raisonnables pour un système d'exploitation moderne à usage général.

Secret	Algorithme	Génération	Stockage	Rotation
Mot de passe user	BCrypt coût 12	Utilisateur (contrainte complexité)	Hash BCrypt en BDD	Sur demande ou expiration 90j/30j
Clé JWT	HS256, ≥256 bits	openssl rand -base64 32	Variable d'env \${JWT_SECRET}	Redéploiement — invalide tous tokens
Clé AES	AES-256-GCM	openssl rand -base64 32	Variable d'env \${AES_KEY}	Redéploiement + rechiffrement BDD
Clé HMAC	HMAC-SHA256	openssl rand -base64 32	Variable d'env \${HMAC_SECRET}	Redéploiement — invalide vérif. chaîne
OTP connexion	SecureRandom 6 chiffres	Serveur à chaque connexion	Hash temporaire BDD	Usage unique, exp. 10 min
Token réinit MDP	UUID 8 chars majuscules	Serveur sur demande	Hash temporaire BDD	Usage unique, exp. 30 min

8. Assurance

Cette section décrit l'ensemble des activités de test et de vérification menées sur le code source soumis, les méthodes utilisées, la couverture atteinte et la fréquence d'exécution.

8.1 Tests unitaires

La suite de tests automatisés comprend 18 tests unitaires répartis dans deux classes de test : AuthServiceTest.java (12 tests) et AesEncryptorTest.java (6 tests). Ces tests utilisent le framework JUnit 5 et Mockito pour les mocks.

AuthServiceTest.java couvre cinq domaines fonctionnels via des classes imbriquées. Le bloc Inscription (3 tests) vérifie le rejet d'un email déjà utilisé, l'enregistrement réussi d'un compte, et le rejet d'un mot de passe ne respectant pas les règles de complexité. Le bloc Complexité du mot de passe (4 tests) vérifie le rejet d'un mot de passe trop court, le rejet d'un mot de passe sans chiffre, le rejet d'un mot de passe sans caractère spécial, et l'acceptation d'un mot de passe valide. Le bloc Verrouillage de compte (3 tests) vérifie qu'un compte verrouillé est rejeté sans vérification du mot de passe, que le verrou expiré est levé automatiquement, et qu'un verrou actif bloque l'accès avec le bon message. Le bloc Expiration de mot de passe (3 tests) vérifie l'expiration après 90 jours pour un client, après 30 jours pour un administrateur, et l'absence d'expiration pour un mot de passe récent. Le bloc Réinitialisation (2 tests) vérifie la réponse neutre anti-énumération et le rejet d'un token expiré.

AesEncryptorTest.java couvre le module cryptographique en 6 tests : cohérence du chiffrement/déchiffrement (round-trip), randomisation de l'IV (deux chiffrements du même texte produisent des résultats différents), absence du texte clair dans le ciphertext, rejet d'un ciphertext altéré (intégrité du tag GCM), chiffrement d'une chaîne vide, et chiffrement d'un numéro de compte typique au format SB-XXXX1234.

8.2 Tests d'intégration

L'intégration entre le frontend React et le backend Spring Boot a été testée manuellement sur l'ensemble des flux utilisateur. Des tests de bout en bout ont été conduits via le navigateur pour chaque fonctionnalité livrée dans les Sprint Alpha et Beta. Des tests directs sur l'API via curl ont également été effectués pour vérifier les comportements hors interface (en-têtes de sécurité, codes HTTP, réponses d'erreur). Les incompatibilités détectées lors de l'intégration (format des réponses API, extraction du token JWT, doubles appels) ont été corrigées et les corrections ont été intégrées dans les tests de non-régression manuels.

8.3 Analyse statique — Semgrep

L'outil d'analyse statique retenu par l'équipe est Semgrep, qui analyse le code source directement (contrairement à SpotBugs qui analyse le bytecode). Les règles de sécurité Java activées couvrent les injections SQL, les vulnérabilités XSS, les algorithmes cryptographiques faibles (MD5, DES, ECB), la gestion incorrecte des secrets et les SSRF.

Résultats : Semgrep n'a détecté aucune vulnérabilité de sécurité critique dans le code produit par l'équipe. Les alertes générées concernent des patterns liés à l'injection Spring et aux proxies Hibernate, documentés et justifiés. Aucune alerte relative à un algorithme faible, une injection ou une fuite de secret n'a été détectée.

8.4 Tests de sécurité spécifiques

Au-delà des tests fonctionnels, des tests ciblés ont été conduits sur les exigences de sécurité. Le mécanisme de verrouillage a été testé en soumettant délibérément trois mots de passe incorrects consécutifs et en vérifiant le blocage du quatrième essai. Le mécanisme de déconnexion par inactivité a été testé en laissant la session ouverte sans interaction pendant plus de 10 minutes. La réponse anti-énumération du flux de réinitialisation a été vérifiée en soumettant un email inexistant et en confirmant que la réponse est identique à celle d'un email existant. La protection contre la réutilisation des OTP a été vérifiée en tentant d'utiliser un code déjà consommé.

8.5 Revue de code

Une revue de code complète a été conduite après la livraison initiale du Sprint Beta. Cette revue a permis d'identifier sept écarts entre le rapport et l'implémentation effective, documentés dans la section 9 du rapport Sprint Beta. Les corrections ont été appliquées et constituent le livrable final. La revue a été effectuée par une lecture systématique de chaque fichier Java modifié et de chaque composant React livré, en croisant le code avec les objectifs fonctionnels et les exigences du cours.

8.6 Couverture de code et régression

La couverture de code automatisée n'a pas été mesurée formellement dans ce projet en raison des contraintes de temps. Les 18 tests unitaires couvrent les chemins critiques des modules AuthService et AesEncryptor, qui concentrent la quasi-totalité de la logique de sécurité. Les tests manuels de bout en bout couvrent l'intégralité des 17 fonctionnalités du Sprint Beta. Les tests de régression sont exécutés manuellement à chaque modification significative du backend avant livraison.

Activité de test	Portée	Méthode	Résultat
Tests unitaires JUnit 5	AuthService (12 tests), AesEncryptor (6 tests)	Automatisé — mvn test	18/18 passants
Analyse statique Semgrep	Code source Java et React	Automatisé — semgrep -- config=p/java	0 vulnérabilité critique
Tests d'intégration	17 fonctionnalités Sprint Beta	Manuel — navigateur + curl	Toutes validées
Tests de sécurité ciblés	Verrouillage, inactivité, anti- énumération, OTP	Manuel	Tous validés
Revue de code	Ensemble des fichiers modifiés	Lecture croisée code / rapport	7 écarts identifiés et corrigés

9. Conclusion

Ce document de rapport final démontre que SecureBank répond à l'ensemble des sept exigences fondamentales de sécurité définies par le cours. L'autorisation est assurée par une combinaison de RBAC Spring Security et de contrôle DAC côté service. L'authentification repose sur trois facteurs successifs (BCrypt, OTP, JWT) renforcés par quatre mécanismes de protection supplémentaires. L'audit est garanti par un journal à chaînage HMAC-SHA256 tamper-evident. La confidentialité est assurée par TLS en transit et AES-256-GCM au repos. L'intégrité est maintenue par les transactions ACID, le chaînage HMAC et la validation systématique des entrées. La gestion des secrets est conforme aux bonnes pratiques DevSecOps avec externalisation dans des variables d'environnement. L'assurance est démontrée par 18 tests unitaires automatisés, une analyse statique Semgrep sans alerte critique, et une revue de code complète ayant conduit à sept corrections.

Ce projet illustre concrètement qu'une approche Security by Design, où la sécurité est une contrainte de conception première plutôt qu'un ajout ultérieur, est réalisable dans le cadre d'un projet universitaire de taille réduite. Le journal d'audit à chaînage HMAC et le chiffrement AES-256-GCM illustrent en particulier comment des techniques cryptographiques avancées peuvent être appliquées à des problèmes pratiques d'intégrité et de confidentialité dans une application réelle.